

SECURE ENTERPRISE PORTAL

Employee Leave Management System

A Flask-Based Web Application for Streamlined HR Operations. Built with a responsive UI, dynamic database calculations, and role-based access configurations.

DEVELOPED BY: **K Chandravaradhan Reddy**

PROJECT DATE: **June 2026**

Addressing Legacy HR Friction

⚠️ Legacy Management Problems

- **Calculation Liabilities:** Manual leave balance updates are error-prone and spark administrative tension.
- **Slow Processing:** Multi-step approvals routed via physical forms or back-and-forth email chains leak time.
- **Zero Visibility:** Absence of unified balance metrics reduces clear real-time transparency for staff.

✂️ The Modern Automated Purpose

- **Accurate Balances:** Reliable database-driven operations protect balances from manual tracking slips.
- **High-Efficiency:** Direct portal pipelines quickly connect employees to administrative decision layers.
- **Live Transparency:** Renders a clear and complete visual profile of remaining dynamic balances.

System Scope & Objectives



Automated Workflows

Completely digitize simple leave requests, submission forms, and direct administrative approval queues.



Secure Domain Gates

Enforce strict corporate parameters restricting user registration to whitelist business domains.



Universal UI Navigation

Provide robust UI and structured navigation that fluidly adjusts onto diverse dynamic devices and screens.

The Secure Enterprise Stack



Backend

Constructed on Python 3.8+ leveraging the flexible Python Flask 3.0 framework hierarchy.



Database

Powered by SQLite with declarative data models mapped directly through Flask-SQLAlchemy.



Security

Werkzeug cryptographic password hashing and highly secure, dynamic browser cookie sessions.



Deployment

Robust Gunicorn WSGI web servers continuously deployed and live-hosted on Render.com.

System Architecture Pattern

MVC Paradigm Separation

- **Models Layer:** Explicit database structures and relationships mapped cleanly via SQLAlchemy modules.
- **Templates View:** Renders secure, dynamic browser-side interfaces relying on standard Jinja2 templates.
- **Controller Routes:** Core application routes and backend processing rules defined in app.py.

System Pipeline Pattern

- **Web Client:** HTML5 / CSS3 / Vanilla JS rendering interactive elements.
- **Business Engine:** Flask Core coordinates permissions, business processes, and request rules.
- **Data Layer:** SQLite backend securely organizing employee and system records.

Relational Database Schemas

Table Name	Columns Schema Details	Role & Structural Logic
Settings	id, key (String), value (Text)	Stores global system limits and approved corporate email domains.
Admins	id, name, email, password	Houses hashed admin and manager access parameters.
Employees	id, emp_id, allowance_balances, status	Tracks user allowances (Casual: 12, Sick: 10, Earned: 15).
Leave Requests	id, employee_id (FK), dates, status, remarks	Manages requests with lifecycle status tags (Pending/Approved/Rejected).

Portal Feature Matrix

Employee Capabilities

- **Accrual Trackers:** Real-time status screens presenting remaining leave banks visually.
- **Validated Requests:** Automatic day calculations preventing accidental calculation submissions.
- **Audit Timelines:** Access complete request tracking from creation to executive approval.

Administrative Controls

- **Configuration Setup:** Simple first-run systems to define system parameters dynamically.
- **Registry Gatekeeping:** Control domain-restricted portal access for verified entries.
- **Action Command Inbox:** Efficient central panel to quickly approve or decline leave request tasks.

Active Security & Safeguards



Cryptographic Protections

Integrates PBKDF2 hashing routines inside core password storage layouts to maximize backend account security.



Middleware Guardrails

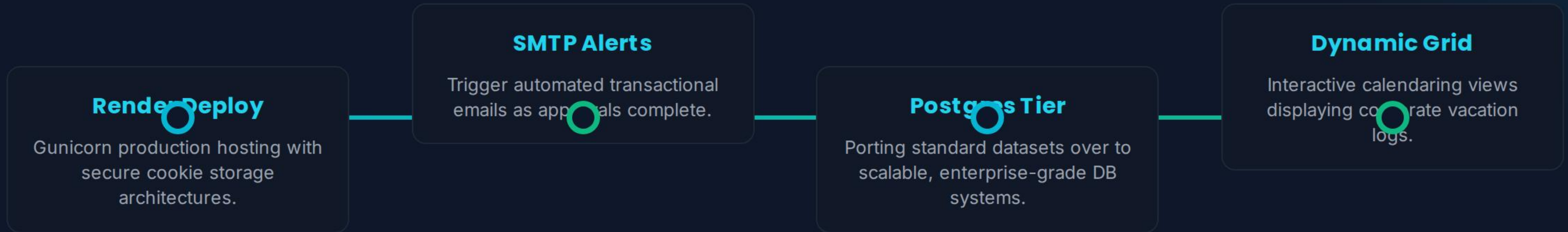
Authentication wrapper layers continuously check authorizations on every server request to stop route vulnerabilities.



SQL Parameterization

Strictly structured, parameterized queries block dynamic SQL command exploits across database portals.

Launch Pipeline & Roadmap



Questions & Discussion

A clean, secure, and production-ready solution built to automate manual workflows, prevent human error, and maximize corporate coordination.



GITHUB SOURCE CODE REPOSITORY

github.com/k-chandravardhanreddy/Employee_Leave_management